

O documento "PDF_módulo 1_git.pdf" apresenta os fundamentos do controle de versão, estabelecendo o modelo mental necessário para utilizar o Git de forma profissional. A base da ferramenta foca na rastreabilidade, integridade de dados e autonomia no desenvolvimento.

Teoria e Modelo Mental do Git

- **Evolução do Controle de Versão:** O controle manual (copiar pastas como `projeto-v1`, `projeto-final`) falha por não preservar a autoria, a motivação das mudanças e a facilidade de recuperação. O Git resolve isso como um sistema **distribuído**, onde cada clone possui o histórico completo do projeto, permitindo o trabalho offline e redundância.
-
- **Snapshots vs. Deltas:** O Git não guarda apenas uma lista de linhas adicionadas ou removidas a cada salvamento; ele captura um *snapshot* (instantâneo lógico) do projeto inteiro. Se um arquivo não foi alterado, o Git reutiliza a referência existente, mantendo a eficiência.
-
- **Banco de Objetos e Hashes:** O Git funciona como um banco de dados endereçável por conteúdo. Cada objeto recebe um identificador Hash SHA-1 (40 caracteres hexadecimais). Qualquer mudança no arquivo, por menor que seja, gera um novo hash, garantindo a integridade e tornando os commits imutáveis.
-
- **Objetos Internos:**
 - **Blob:** Armazena apenas o conteúdo do arquivo, sem nome ou caminho.
 -
 - **Tree:** Estrutura de diretórios que mapeia nomes e caminhos para os blobs.
 -
 - **Commit:** Aponta para uma *Tree*, guarda metadados (autor, data, mensagem) e faz referência ao commit pai.
 -
 - **Tag:** Referência nomeada para marcar uma versão estável.
 -
- **Grafo Acíclico Direcionado (DAG):** O histórico do Git forma uma cadeia onde cada commit aponta para o seu ancestral, permitindo um desenvolvimento não linear e navegação eficiente.
-

As Três Árvores e Estados dos Arquivos

O fluxo de trabalho local do Git opera através de três camadas principais:

1. **Working Directory (Diretório de Trabalho):** A camada visível onde os arquivos são editados. Arquivos novos estão no estado **Untracked** (desconhecidos pelo Git) e arquivos editados ficam como **Modified**.
- 2.

3. **Staging Area / Index (Área de Preparação):** A camada de curadoria. Ao usar o comando de adição, o estado exato do arquivo naquele momento é preparado para o próximo commit, ficando no estado **Staged**.
- 4.
5. **Repository (Repositório):** A memória persistente oculta na pasta `.git`. Ao finalizar a curadoria, os arquivos entram no histórico oficial no estado **Committed**.
- 6.

Comandos Principais e Casos de Uso

- `git init`
- - *Uso:* Transforma uma pasta comum em um repositório Git, criando o diretório oculto `.git` que atua como banco de dados local.
 -
- `git config --global user.name "Nome" e user.email "Email"`
- - *Uso:* Define a identidade do autor. Essencial antes de iniciar o trabalho para garantir que todos os commits tenham a autoria correta, permitindo rastreabilidade no trabalho em equipe.
 -
- `git status`
- - *Uso:* Diagnosticar o estado atual dos arquivos (Untracked, Modified, Staged) comparando o diretório de trabalho com a referência ativa (`HEAD`).
 -
- `git add <arquivo>`
- - *Uso:* Move uma alteração do *Working Directory* para a *Staging Area*. Ideal para selecionar alterações específicas (curadoria) que farão parte da próxima unidade de histórico, sem incluir arquivos incompletos.
 -
- `git commit`
- - *Uso:* Captura os arquivos da *Staging Area* e cria um novo objeto de snapshot no repositório, acompanhado de uma mensagem clara. Usado para registrar uma evolução lógica e recuperável do projeto.
 -
- `git log / git log --oneline graph`
-

- *Uso:* Consulta o histórico de commits, autoria e mensagens. A variação em grafo permite visualizar as setas de parentesco (DAG) e bifurcações do projeto.
-
- `git diff`
-
- *Uso:* Mostra linha a linha o que foi modificado em um arquivo antes de prepará-lo para commit.
-
- `git help <comando>`
-
- *Uso:* Abre a documentação técnica oficial de um comando para descobrir parâmetros ou sintaxe.
-

Erros Comuns de Usuários

- **Tratar o Git apenas como Backup:** Acreditar que o Git serve apenas para guardar cópias na nuvem, ignorando que sua função primária é preservar a história estruturada, autoria e relações entre os estados.
-
- **Confusão de Salvamento Automático:** Acreditar que editar e salvar um arquivo no editor de código já cria uma versão no histórico (esquecendo de passar pelo `git add` e `git commit`).
-
- **Edição manual do diretório `.git`:** Tentar alterar manualmente os arquivos dentro da pasta oculta `.git` (como `objects`, `refs`, `HEAD`), o que corrompe o banco de dados do repositório.
-
- **Não configurar Line Endings (Quebras de linha):** Esquecer de normalizar o formato `CRLF` (Windows) e `LF` (Mac/Linux) com `core.autocrlf`, o que faz o Git registrar alterações falsas em arquivos inteiros apenas pela quebra de linha, gerando ruído colaborativo.
-
- **Comitar arquivos sensíveis:** Incluir senhas, dados pessoais ou arquivos temporários no commit. Como os commits são imutáveis e baseados em hash, remover esses dados depois é complexo.
-
- **Acreditar que `git add` é definitivo:** Pensar que adicionar o arquivo à *Staging Area* já registra o histórico final, quando na verdade é apenas uma área de rascunho/preparação.
-

Glossário

- **Git:** Sistema distribuído de controle de versão.
-
- **GitHub:** Plataforma de hospedagem e colaboração baseada em repositórios Git.

-
- **Commit:** Registro histórico de um snapshot preparado.
-
- **Snapshot:** Estado lógico do projeto em determinado momento.
-
- **Hash:** Identificador calculado a partir do conteúdo.
-
- **DAG:** Grafo Acíclico Direcionado usado para representar histórico.
-
- **Working Directory:** Área visível onde os arquivos são editados.
-
- **Staging Area:** Área de preparação do próximo commit.
-
- **Repository:** Histórico persistente de commits.
-
- **HEAD:** Referência para o estado ativo.
-
- **Blob:** Objeto Git que armazena conteúdo de arquivo.
-
- **Tree:** Objeto Git que mapeia nomes e caminhos para objetos.
-

Tabela de Comandos Utilizados

Comando Git	Descrição / Efeito
<code>git --version</code>	Verifica a instalação e a versão atual do Git.
<code>git init</code>	Inicializa um novo repositório local, criando o diretório <code>.git</code> .
<code>git config --global user.name</code>	Configura o nome do autor associado aos commits.

<code>git config --global user.email</code>	Configura o e-mail do autor associado aos commits.
<code>git config --list --show-origin</code>	Exibe todas as configurações aplicadas e suas origens (local, global, system).
<code>git config --global core.autocrlf</code>	Configura o tratamento de quebras de linha entre sistemas operacionais diferentes.
<code>git status</code>	Exibe o estado da árvore de trabalho e da área de preparação.
<code>git add <arquivo></code>	Adiciona o conteúdo modificado ao índice (<i>Staging Area</i>).
<code>git commit</code>	Registra o snapshot preparado no repositório.
<code>git restore --staged</code>	Remove um arquivo da área de preparação, devolvendo-o ao estado modificado sem perder o conteúdo.

<code>git diff</code>	Exibe as diferenças linha a linha entre o diretório de trabalho e o índice.
<code>git log</code>	Exibe o histórico de commits.
<code>git log --oneline graph</code>	Exibe o histórico de commits de forma resumida e com visualização do grafo.
<code>git help <comando></code>	Abre o manual de documentação oficial de um comando específico.
<code>git hash-object / git cat-file -p</code>	Comandos internos para inspecionar hashes e objetos diretamente no banco de dados do Git.

Aqui está um quiz completo com 30 questões em extenso sobre o Módulo 1 do Git (Fundamentos, Instalação e Modelo Mental), elaborado com base no conteúdo e no resumo do módulo.

Quiz de Fixação: Módulo 1 – Fundamentos, Instalação e Modelo Mental do Git

Questões

1. Qual é a limitação fundamental do controle de versão manual (como salvar cópias `projeto-v1`, `projeto-final`) em relação a um VCS estruturado?

- A) Não permite abrir os arquivos em mais de um editor de texto ao mesmo tempo.
- B) Ocupa muito espaço em disco, pois impede a compactação de arquivos ZIP.
- C) Não preserva a autoria, a motivação das mudanças nem a reprodutibilidade do histórico.
- D) Exige obrigatoriamente uma conexão constante com a internet para salvar cópias.

2. O Git é classificado como um Sistema de Controle de Versão Distribuído (DVCS). O que isso significa na prática?

- A) O histórico do projeto fica armazenado apenas em um servidor centralizado na nuvem.
- B) Cada clone possui uma cópia completa de todo o banco de dados e do histórico do repositório.
- C) As alterações só podem ser registradas no histórico local se houver autorização do servidor remoto.
- D) O histórico é fragmentado, onde cada desenvolvedor guarda apenas as suas próprias linhas de código.

3. Qual foi a motivação histórica primária para a criação do Git por Linus Torvalds em 2005?

- A) Substituir o SVN após a descontinuação de seu suporte oficial pela Apache.
- B) A necessidade urgente de substituir o BitKeeper no projeto do kernel Linux devido a questões de licenciamento.
- C) Criar uma plataforma web comercial para hospedagem de código-fonte aberta ao público.
- D) Padronizar o desenvolvimento de softwares escritos exclusivamente na linguagem C.

4. Qual é a relação conceitual correta entre Git e GitHub?

- A) Git é a plataforma de rede social e GitHub é o comando que roda no terminal do computador.
- B) Git e GitHub são exatamente a mesma ferramenta com nomes comerciais diferentes.
- C) Git é o sistema local/distribuído de controle de versão; GitHub é uma plataforma web construída sobre o Git.
- D) GitHub é o banco de dados interno da pasta `.git`, enquanto o Git é a interface gráfica do programa.

5. Qual das opções representa uma desvantagem dos Sistemas de Controle de Versão Centralizados (como CVS e Subversion) em relação aos Distribuídos?

- A) Impossibilidade de definir controle de acesso e permissões administrativas.
- B) Dependência do servidor central: se ele estiver indisponível, os commits ficam bloqueados.
- C) Falta de suporte a arquivos de texto simples em projetos colaborativos.
- D) Lentidão para executar comandos exclusivamente locais no terminal do usuário.

6. Como o modelo de dados do Git registra a evolução dos arquivos a cada commit?

- A) Salvando apenas a lista de linhas adicionadas e removidas (deltas).
- B) Duplicando fisicamente todos os arquivos do computador a cada salvamento.
- C) Capturando snapshots (instantâneos lógicos) do estado completo do projeto e reutilizando referências para o que não mudou.
- D) Criando cópias compactadas em formato `.zip` organizadas por data e hora dentro do sistema operacional.

7. O que é um Hash SHA-1 dentro do Git e como ele se estrutura?

- A) Um número sequencial automático (1, 2, 3...) gerado a cada novo registro no repositório.
- B) Uma sequência hexadecimal de 40 caracteres calculada estritamente a partir do conteúdo do objeto.
- C) Uma chave de criptografia privada que só o servidor do GitHub consegue decodificar.
- D) Um rótulo de texto editável criado pelo usuário para dar nome ao repositório.

8. O que ocorre com o identificador Hash SHA-1 de um arquivo se um único espaço ou caractere for alterado em seu conteúdo?

- A) O hash permanece idêntico, pois o nome e o caminho do arquivo não foram alterados.
- B) O hash é apagado e o Git exige a recriação manual do repositório.
- C) O hash muda completamente, alterando a identidade do objeto.
- D) O hash ganha apenas um sufixo numérico adicional (ex: `-1`) no final da sequência.

9. Por que se afirma que os commits no Git são imutáveis por padrão?

- A) Porque é impossível apagar o Git do sistema operacional depois de instalado.
- B) Porque qualquer alteração nos metadados ou no conteúdo do commit resulta na geração de um novo hash SHA-1.
- C) Porque o Git trava a edição de arquivos no sistema operacional imediatamente após o salvamento.
- D) Porque as mensagens de commit são salvas em um hardware de leitura rápida que não aceita regravações.

10. Qual princípio de projeto do Git permite que operações como `git status` e `git log` sejam executadas instantaneamente sem acesso à rede?

- A) Descentralização e execução estritamente local de operações.
- B) Dependência de servidores espelho (*mirrors*) no ecossistema cloud.
- C) Compactação de dados por Inteligência Artificial.
- D) Uso exclusivo de arquivos temporários na memória RAM.

11. No banco de dados interno do Git, qual é a função do objeto do tipo Blob?

- A) Armazenar a estrutura de diretórios, nomes de arquivos e permissões de acesso.
- B) Armazenar exclusivamente o conteúdo em bytes de um arquivo, sem seu nome ou caminho original.
- C) Conectar o autor e a mensagem descritiva ao snapshot do projeto.
- D) Apontar a localização do servidor remoto no GitHub.

12. Qual é a responsabilidade do objeto do tipo Tree no modelo de objetos do Git?

- A) Mapear os nomes de arquivos, caminhos e permissões, apontando para blobs ou outras trees.
- B) Guardar as senhas e chaves SSH do usuário em formato criptografado.
- C) Registrar apenas as mensagens explicativas dos erros do sistema.
- D) Salvar as configurações de visualização do terminal do usuário.

13. Quais metadados essenciais estão contidos dentro do objeto Commit?

- A) Apenas a lista de arquivos alterados no formato PDF.
- B) Ponteiro para a tree raiz, dados de autoria (nome/e-mail), data, hora, mensagem e referência ao commit pai.
- C) O IP da máquina, o sistema operacional utilizado e a chave de licença do software.
- D) Exclusivamente o link direto para o repositório no GitHub.

14. O histórico do Git forma uma estrutura matemática conhecida como DAG. O que significa essa sigla?

- A) *Data Architecture Group* (Grupo de Arquitetura de Dados).
- B) *Directed Acyclic Graph* (Grafo Acíclico Direcionado).
- C) *Distributed Automatic Git* (Git Automático Distribuído).
- D) *Digital Authentication Gate* (Portal de Autenticação Digital).

15. O que representa a referência HEAD no ecossistema do Git?

- A) O primeiro commit historicamente realizado ao criar a conta no GitHub.
- B) Um ponteiro que indica o estado/branch ativo no momento.
- C) O arquivo de configuração global que gerencia as permissões de administrador.
- D) Um comando usado exclusivamente para deletar branches corrompidas.

16. O que é a camada Working Directory (Diretório de Trabalho)?

- A) O repositório remoto armazenado nos servidores da nuvem.
- B) A área visível no disco onde o desenvolvedor edita, cria e manipula seus arquivos diretamente.
- C) A área secreta onde ficam salvos os hashes criptografados dos arquivos deletados.

D) O banco de dados do repositório localizado dentro do diretório `.git`.

17. Qual é o papel da Staging Area (também chamada de *Index*) no fluxo das três árvores?

A) Servir como camada de curadoria onde se seleciona exatamente o conteúdo preparado para compor o próximo commit.

B) Apagar automaticamente os arquivos temporários do sistema operacional.

C) Enviar as alterações diretamente para o servidor do GitHub sem passar pelo repositório local.

D) Armazenar os backups diários da máquina do desenvolvedor.

18. O que significa quando o comando `git status` relata que um arquivo está no estado Untracked?

A) O arquivo foi excluído do disco rígido e precisa ser restaurado.

B) O arquivo sofreu alterações, mas já está pronto para o commit no repositório.

C) O arquivo existe no diretório de trabalho, mas ainda não é acompanhado pelo controle do Git.

D) O arquivo contém vírus e foi bloqueado pelo sistema de integridade do Git.

19. Como um arquivo passa do estado Modified no Working Directory para o estado Staged?

A) Salvando o arquivo no editor de código com `Ctrl + S`.

B) Executando o comando `git add <arquivo>`.

C) Executando o comando `git commit` no terminal.

D) Fechando e reabrindo o terminal do Git Bash.

20. O estado Committed indica que:

A) As alterações foram enviadas e aprovadas pela equipe no repositório do GitHub.

B) Os arquivos foram salvos temporariamente na memória RAM para testes rápidos.

C) O snapshot do conteúdo preparado foi gravado de forma persistente no banco de dados local `.git`.

D) O código está livre de erros de compilação ou sintaxe.

21. Qual comando é utilizado para transformar um diretório comum do sistema em um repositório Git local?

A) `git create`

B) `git init`

C) `git start`

D) `git new`

22. Qual é a utilidade do comando `git status`?

A) Exibir o manual de ajuda e a lista de todos os comandos disponíveis.

B) Diagnosticar o estado atual dos arquivos comparando as três árvores (Working Directory, Staging Area e HEAD).

C) Baixar as últimas atualizações registradas no servidor do GitHub.

D) Alterar a chave de e-mail e nome do usuário do sistema.

23. Caso você tenha preparado um arquivo com `git add`, mas queira retirá-lo da Staging Area sem perder suas alterações no arquivo, qual comando deve usar?

A) `git remove`

B) `git restore --staged <arquivo>`

C) `git delete`

D) `git status --clean`

24. Qual o efeito da execução do comando `git log --oneline --graph`?

A) Exibir o histórico de commits de forma resumida em linha única e com a representação visual das ramificações do grafo.

B) Limpar todo o histórico do repositório para economizar memória do computador.

C) Abrir uma janela de navegador exibindo os gráficos estatísticos do GitHub.

D) Exibir apenas os erros ocorridos durante o processo de instalação do Git.

25. O comando `git diff` é utilizado para:

A) Mostrar a diferença linha a linha entre o Working Directory e a Staging Area.

B) Comparar a velocidade de execução da internet com o servidor remoto.

C) Renomear as pastas do projeto de forma automática.

D) Apagar commits antigos que estejam em duplicidade.

26. A respeito dos escopos de configuração do Git (`system`, `global` e `local`), qual possui a maior prioridade de sobrescrita?

A) `system`

B) `global`

C) `local`

D) Todos possuem exatamente a mesma prioridade sem hierarquia.

27. Por que a configuração de tratamento de quebras de linha (`core.autocrlf`) é crítica em projetos colaborativos?

A) Porque impede que o código seja copiado por usuários de sistemas operacionais concorrentes.

B) Porque evita que diferenças sutis de fim de linha entre Windows (CRLF) e Linux/macOS (LF) marquem um arquivo inteiro como modificado.

C) Porque altera automaticamente a linguagem de programação de acordo com o sistema operacional instalado.

D) Porque criptografa os caracteres especiais do texto antes do envio ao repositório remoto.

28. Qual a importância ética e operacional de configurar corretamente `user.name` e `user.email` antes de fazer commits?

A) Sem isso, o Git não consegue compilar programas em linguagens como C ou Java.

B) Permite a correta atribuição de autoria, rastreabilidade e responsabilidade técnica de cada alteração no histórico.

C) É um requisito pago e obrigatório para obter a licença de uso do Git.

D) É a única forma de habilitar a interface gráfica do programa no Windows.

29. Por que a edição manual direta dos arquivos contidos na pasta oculta .git (como `objects`, `refs` ou `HEAD`) é considerada uma má prática grave?

A) Porque desinstala automaticamente o terminal do computador.

B) Porque pode corromper o banco de dados interno e inutilizar a integridade do histórico do repositório.

C) Porque apaga os arquivos originais salvos na pasta de documentos do sistema.

D) Porque expõe o endereço IP da máquina na rede pública.

30. O que torna o registro inadvertido de dados sensíveis (senhas, tokens de API ou dados pessoais) em um commit um risco grave de segurança?

A) O Git bloqueia o computador e impede a inicialização do sistema até que a senha seja trocada.

B) Os commits são imutáveis e persistem no banco de objetos do repositório, permanecendo recuperáveis no histórico.

C) O Git envia as senhas por e-mail automaticamente para todos os administradores cadastrados.

D) Dados sensíveis impedem a execução de comandos básicos como `git status` e `git log`.

Gabarito Comentado

1. **1C – Justificativa:** Cópias manuais de pasta perdem o registro de quem fez a alteração, quando foi feita e por qual motivo. O VCS automatiza esse rastreamento com segurança.
- 2.
3. **2B – Justificativa:** A marca registrada dos sistemas distribuídos (DVCS) é que cada cópia clonada é um repositório completo com todo o histórico e banco de dados, não dependendo do servidor central para operações locais.
- 4.
5. **3B – Justificativa:** Linus Torvalds criou o Git em 2005 para atender às necessidades do kernel Linux quando o uso gratuito da ferramenta proprietária BitKeeper foi revogado.
- 6.
7. **4C – Justificativa:** O Git é a ferramenta local de linha de comando responsável pelo versionamento; o GitHub é um serviço em nuvem para hospedar e colaborar em repositórios Git.

- 8.
9. **5B – Justificativa:** Em sistemas centralizados (como SVN), o servidor central é um ponto único de falha; se estiver indisponível, os desenvolvedores ficam impossibilitados de registrar novos commits.
- 10.
11. **6C – Justificativa:** O Git registra *snapshots* (instantâneos lógicos) do estado completo do projeto a cada commit. Arquivos não modificados têm suas referências reutilizadas sem duplicar os dados.
- 12.
13. **7B – Justificativa:** O SHA-1 é uma chave hexadecimal de 40 caracteres obtida via cálculo matemático sobre o conteúdo e metadados de cada objeto no Git.
- 14.
15. **8C – Justificativa:** Como o hash é calculado diretamente a partir do conteúdo dos bytes, qualquer pequena modificação altera o valor final gerado.
- 16.
17. **9B – Justificativa:** A alteração de qualquer informação num commit (conteúdo, data, mensagem) gera obrigatoriamente um novo hash SHA-1. O commit original permanece intocado.
- 18.
19. **10A – Justificativa:** Sendo um sistema distribuído, quase todas as consultas de histórico e status são processadas localmente a partir do diretório `.git` do computador, dispensando a rede.
- 20.
21. **11B – Justificativa:** O *Blob* é o objeto básico de conteúdo; ele guarda apenas os bytes do arquivo de forma agnóstica a nomes ou estruturas.
- 22.
23. **12A – Justificativa:** A *Tree* é a estrutura do Git que reflete diretórios e mapeia nomes de arquivos, permissões e aponta para os *Blobs* ou sub-trees correspondentes.
- 24.
25. **13B – Justificativa:** Um objeto de *Commit* une o ponteiro da *Tree* raiz com os metadados de quem salvou (autor/commiter), quando, por qual motivo (mensagem) e seu commit antecedente (pai).
- 26.
27. **14B – Justificativa:** DAG significa *Directed Acyclic Graph* (Grafo Acíclico Direcionado). Os ponteiros vão do commit filho para o pai, impedindo ciclos fechados.
- 28.
29. **15B – Justificativa:** O `HEAD` é uma referência especial que aponta para o estado atual em que o diretório de trabalho se encontra no histórico.
- 30.
31. **16B – Justificativa:** O *Working Directory* (Diretório de Trabalho) é o ambiente de arquivos do sistema operacional onde as modificações e codificações acontecem ativamente.
- 32.
33. **17A – Justificativa:** A *Staging Area* (Index) funciona como uma área de preparação seletiva. Nela você decide quais modificações farão parte da unidade atômica do próximo commit.

- 34.
35. **18C – Justificativa:** O estado *Untracked* sinaliza que o arquivo é recente ou nunca foi adicionado ao índice do Git para rastreamento.
- 36.
37. **19B – Justificativa:** O comando `git add <arquivo>` captura o estado atual do arquivo no diretório de trabalho e o coloca no estado *Staged* na área de preparação.
- 38.
39. **20C – Justificativa:** O estado *Committed* garante que o snapshot das alterações contidas na *Staging Area* foi gravado de forma persistente no repositório local (`.git`).
- 40.
41. **21B – Justificativa:** O comando `git init` inicializa a estrutura interna do repositório e cria o diretório `.git` na pasta atual.
- 42.
43. **22B – Justificativa:** O `git status` fornece a leitura rápida do estado dos arquivos, indicando quais estão em *Untracked*, *Modified* ou *Staged*.
- 44.
45. **23B – Justificativa:** O comando `git restore --staged <arquivo>` remove o arquivo da área de preparação (*Staging Area*), mantendo intactas as alterações feitas no arquivo no diretório de trabalho.
- 46.
47. **24A – Justificativa:** O parâmetro `--online` reduz cada commit a uma linha com seu hash curto, e `--graph` adiciona uma representação ascii das ramificações do histórico.
- 48.
49. **25A – Justificativa:** O `git diff` expõe exatamente quais linhas foram adicionadas, removidas ou alteradas no *Working Directory* que ainda não foram marcadas com `git add`.
- 50.
51. **26C – Justificativa:** O escopo `local` tem a maior prioridade e sobrescreve as configurações de nível `global` e `system` para aquele repositório em específico.
- 52.
53. **27B – Justificativa:** Diferenças entre sistemas (Windows usa CRLF, Linux/macOS usam LF) podem fazer com que todo o arquivo pareça alterado. O `core.autocrlf` normaliza essas quebras de linha para evitar ruídos no histórico.
- 54.
55. **28B – Justificativa:** Em projetos e equipes reais, a identidade associada aos commits garante a rastreabilidade e a transparência de quem introduziu cada alteração.
- 56.
57. **29B – Justificativa:** O diretório `.git` é gerido pelos comandos internos da ferramenta. A modificação manual direta de seus arquivos arrisca corromper os ponteiros do banco de objetos do repositório.
- 58.
59. **30B – Justificativa:** Devido à imutabilidade dos objetos e do histórico por hashes, uma vez que dados sensíveis entram em um commit, eles continuam salvos no banco de objetos e recuperáveis mesmo se apagados num commit posterior.